Лабораторная работа №3

Манякин Степан 6204-010302D

Задание 2

> В пакете functions созданы два класса исключений: FunctionPointIndexOutOfBoundsException - исключение выхода за границы набора точек при обращении к ним по номеру и InappropriateFunctionPointException - исключение, выбрасываемое при попытке добавления или изменения точки функции несоответствующим образом

```
package functions;


public class FunctionPointIndexOutOfBoundsException extends
  IndexOutOfBoundsException {
    public FunctionPointIndexOutOfBoundsException() {
        super("Индекс точки вне диапазона допустимых значений");
    }


    public FunctionPointIndexOutOfBoundsException(String message) {
        super(message);
    }
}
```

```
package functions;


public class InappropriateFunctionPointException extends RuntimeException {
    public InappropriateFunctionPointException() {
        super("Некорректная операция с точкой функции");
    }


    public InappropriateFunctionPointException(String message) {
        super(message);
    }
```

```
}
```

Задание 3

Проведена глубокая модификация класса ArrayTabulatedFunction с добавлением комплексной проверки входных данных и состояний.

В конструкторах создан метод проверки индекса:

```
private void checkIndex(int index) {
    if (index < 0 || index >= pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Index " + index + " out of bounds");
}
```

Вот его реализация в методах setPoint() и setPointX():

```
if ((index > 0 && x <= points[index - 1].getX()) ||
    (index < pointsCount - 1 && x >= points[index + 1].getX()))
    throw new InappropriateFunctionPointException("X out of order");
```

В методе deletePoint():

```
if (pointsCount <= 2)
    throw new IllegalStateException("Cannot delete: less than 3 points");
```

Задание 4

Создан класс с двусвязным циклическим списком:

```
private static class FunctionNode {
    FunctionPoint point;
    FunctionNode next;
    FunctionNode prev;
    FunctionNode(FunctionPoint p) { point = p; }
}
```

Класс FunctionNode был реализован как private static внутренний класс внутри класса LinkedListTabulatedFunction

Обоснование выбора:
Private модификатор выбран потому что FunctionNode является внутренней реализацией связного списка и не должен быть доступен извне класса LinkedListTabulatedFunction
Static модификатор используется так как узлы списка не требуют доступа к полям внешнего класса LinkedListTabulatedFunction
Внутренний класс выбран потому что узлы логически принадлежат списку и не имеют смысла вне его контекста.

```
private final FunctionNode head = new FunctionNode(null);

private int pointsCount;
```

Выделенная голова упрощает алгоритмы работы со списком
Циклическая структура (head.next = head.prev = head) обеспечивает единообразную обработку граничных случаев
Счетчик точек pointsCount позволяет быстро определять размер списка

```
private FunctionNode getNodeByIndex(int index) {

    if (index < 0 || index >= pointsCount)

        throw new FunctionPointIndexOutOfBoundsException("Index " + index + " out of bounds");



    FunctionNode node = head.next;

    for (int i = 0; i < index; i++)

        node = node.next;

    return node;

}
```

Используется последовательный обход от начала списка. Метод не включает голову в нумерацию что соответствует требованию

```
private FunctionNode addNodeToTail() {
    FunctionNode newNode = new FunctionNode(new FunctionPoint(0, 0));

    if (head.next == head) {
        head.next = head.prev = newNode;
        newNode.next = newNode.prev = head;
    } else {
        newNode.next = head;
        newNode.prev = head.prev;
        head.prev.next = newNode;
        head.prev = newNode;
    }

    pointsCount++;
    lastAccessedNode = newNode;
    lastAccessedIndex = pointsCount - 1;
    return newNode;
}
```

Добавлено кэширование последнего доступа и выбор оптимального направления обхода

```
private FunctionNode addNodeByIndex(int index) {
    if (index < 0 || index > pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Index " + index + " out of bounds for insertion");

    FunctionNode newNode = new FunctionNode(new FunctionPoint(0, 0));
```

```
    if (index == pointsCount) {
        return addNodeToTail();
    }

    FunctionNode currentNode = getNodeByIndex(index);
    newNode.prev = currentNode.prev;
    newNode.next = currentNode;
    currentNode.prev.next = newNode;
    currentNode.prev = newNode;

    pointsCount++;

    if (lastAccessedIndex >= index) {
        lastAccessedIndex++;
    }
    lastAccessedNode = newNode;

    return newNode;
}
```

Добавляет новый узел в указанную позицию списка. Если индекс равен количеству точек, добавляет в конец. Выполняет перелинковку соседних узлов и обновляет счетчик точек. Возвращает ссылку на созданный узел.

```
private FunctionNode deleteNodeByIndex(int index) {
    if (index < 0 || index >= pointsCount)
        throw new FunctionPointIndexOutOfBoundsException("Index " + index + " out of bounds");
```

```
    if (pointsCount <= 2)
        throw new IllegalStateException("Cannot delete: less than 3 points");

    FunctionNode nodeToDelete = getNodeByIndex(index);

    nodeToDelete.prev.next = nodeToDelete.next;
    nodeToDelete.next.prev = nodeToDelete.prev;

    if (nodeToDelete == head.next) {
        head.next = nodeToDelete.next;
    }

    pointsCount--;

    if (lastAccessedIndex == index) {
        lastAccessedNode = null;
        lastAccessedIndex = -1;
    } else if (lastAccessedIndex > index) {
        lastAccessedIndex--;
    }

    return nodeToDelete;
}
```

Удаляет узел по указанному индексу. Выполняет перелинковку соседних узлов, обновляет голову при необходимости и корректирует кэш последнего доступа. Возвращает ссылку на удаленный узел.

Задание 5

```
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {
    if (leftX >= rightX) throw new IllegalArgumentException("Left >= Right");
    if (pointsCount < 2) throw new IllegalArgumentException("Points count < 2");

    head.next = head.prev = head;
    this.pointsCount = 0;
    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        addNodeToTail().point = new FunctionPoint(leftX + i * step, 0);
    }
}
```

Данный конструктор создает табулированную функцию с равномерно распределенными точками на интервале [leftX, rightX]. Проверяет корректность границ и количества точек, инициализирует пустой список и заполняет его точками с шагом step, где Y=0.

```
public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
    if (leftX >= rightX) throw new IllegalArgumentException("Left >= Right");
    if (values.length < 2) throw new IllegalArgumentException("Points count < 2");

    head.next = head.prev = head;
    this.pointsCount = 0;
    double step = (rightX - leftX) / (values.length - 1);
    for (int i = 0; i < values.length; i++) {
        addNodeToTail().point = new FunctionPoint(leftX + i * step, values[i]);
    }
}
```

Данный конструктор создает табулированную функцию с равномерной сеткой по X и заданными значениями Y из массива values. Проверяет корректность входных данных и заполняет список точками, используя addNodeToTail().

```
public double getFunctionValue(double x) {
    if (x < getLeftDomainBorder() || x > getRightDomainBorder())
        return Double.NaN;

    FunctionNode node = head.next;
    while (node.next != head) {
        double x1 = node.point.getX();
        double x2 = node.next.point.getX();

        if (x == x1) return node.point.getY();
        if (x > x1 && x < x2) {
            double y1 = node.point.getY();
            double y2 = node.next.point.getY();
            return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
        }
        node = node.next;
    }
    return Double.NaN;
}
```

Вычисляет значение функции в точке x с помощью линейной интерполяции. Находит интервал между узлами, содержащий x, и вычисляет значение по формуле линейной интерполяции. Возвращает NaN если x вне области определения.

```
public void setPoint(int index, FunctionPoint point) throws
  InappropriateFunctionPointException {
```

```
    FunctionNode node = getNodeByIndex(index);
    double x = point.getX();

    if ((node.prev != head && x <= node.prev.point.getX()) ||
        (node.next != head && x >= node.next.point.getX())) {
        throw new InappropriateFunctionPointException("X out of order");
    }
    node.point = new FunctionPoint(point);
}
```

Заменяет точку по указанному индексу. Проверяет что новое значение X сохраняет упорядоченность списка, сравнивая с соседними узлами напрямую. Создает копию точки для защиты инкапсуляции.

```
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException
  {
    FunctionNode node = head.next;
    while (node != head) {
        if (node.point.getX() == point.getX())
            throw new InappropriateFunctionPointException("Duplicate X");
        node = node.next;
    }

    FunctionNode newNode = new FunctionNode(new FunctionPoint(point));
    if (head.next == head) {
        head.next = head.prev = newNode;
        newNode.next = newNode.prev = head;
    } else {
        node = head.next;
```

```
        while (node != head && node.point.getX() < point.getX())
            node = node.next;
        newNode.prev = node.prev;
        newNode.next = node;
        node.prev.next = newNode;
        node.prev = newNode;
    }
    pointsCount++;
}
```

Добавляет новую точку с сохранением упорядоченности по X. Проверяет уникальность координаты X, находит позицию для вставки и использует

```
public void deletePoint(int index) {
    if (pointsCount <= 2)
        throw new IllegalStateException("Cannot delete: less than 3 points");
    FunctionNode node = getNodeByIndex(index);
    node.prev.next = node.next;
    node.next.prev = node.prev;
    if (node == head.next) head.next = node.next;
    pointsCount--;
}
```

Прямое удаление узла с перелинковкой соседей без использования deleteNodeByIndex

```
public double getPointX(int index) { return getNodeByIndex(index).point.getX(); }
public double getPointY(int index) { return getNodeByIndex(index).point.getY(); }
public void setPointY(int index, double y) { getNodeByIndex(index).point.setY(y); }
```

getPointX - возвращает X точки по индексу через прямой доступ к узлу
getPointY - возвращает Y точки по индексу через прямой доступ к узлу
setPointY - устанавливает Y точки по индексу через прямой доступ к узлу

Задание 6

Интерфейс –

```
public interface TabulatedFunction {
    int getPointsCount();
    double getLeftDomainBorder();
    double getRightDomainBorder();
    double getFunctionValue(double x);
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point);
    double getPointX(int index);
    void setPointX(int index, double x);
    double getPointY(int index);
    void setPointY(int index, double y);
    void addPoint(FunctionPoint point);
    void deletePoint(int index);
}
```

Определяет контракт для табулированных функций. Содержит методы для получения количества точек, границ области определения, вычисления значения функции, работы с точками (добавление, удаление, изменение). Обеспечивает единый интерфейс для разных реализаций.

Задание 7

```
import functions.*;
```

```
public class Main {
    public static void main(String[] args) {
        System.out.println("ТЕСТИРОВАНИЕ КЛАССОВ TABULATED
  FUNCTION\n");


        //тестирование ArrayTabulatedFunction
        System.out.println("Тест: ArrayTabulatedFunction");
        testFunction(new ArrayTabulatedFunction(0, 4, new double[]{0, 1, 4, 9, 16}));


        //тестирование LinkedListTabulatedFunction
        System.out.println("\n Тест: LinkedListTabulatedFunction");
        testFunction(new LinkedListTabulatedFunction(0, 4, new double[]{0, 1, 4, 9,
  16}));


        System.out.println("\nВСЕ ТЕСТЫ ЗАВЕРШЕНЫ");
    }


    private static void testFunction(TabulatedFunction func) {
        System.out.println("Тип функции: " + func.getClass().getSimpleName());
        System.out.println();


        //1.проверка вычисления значений
        System.out.println("1 Проверяем значения функции:");
        for (double x = 1; x <= 5; x += 0.5) {
            double y = func.getFunctionValue(x);
            System.out.printf("f(%.1f) = %s\n", x, Double.isNaN(y) ? "NaN" : y);
        }
```

```
    //2.проверка корректности точек
    System.out.println("\n2 Проверяем доступ к точкам:");
    try {
        System.out.println("Количество точек: " + func.getPointsCount());
        for (int i = 0; i < func.getPointsCount(); i++) {
            FunctionPoint p = func.getPoint(i);
            System.out.printf("Точка %d: (%.2f, %.2f)\n", i, p.getX(), p.getY());
        }
    } catch (FunctionPointIndexOutOfBoundsException e) {
        System.out.println("Ошибка доступа к точке: " + e.getMessage());
    }

    //3.проверка ошибок индекса
    System.out.println("\n3 Проверяем ошибки индекса:");
    try {
        func.getPoint(-1);
        System.out.println("Ошибка: ожидалось
FunctionPointIndexOutOfBoundsException, но не было");
    } catch (FunctionPointIndexOutOfBoundsException e) {
        System.out.println("Ловим FunctionPointIndexOutOfBoundsException (-1): " +
e.getMessage());
    }

    try {
        func.getPoint(func.getPointsCount());
        System.out.println("Ошибка: ожидалось
FunctionPointIndexOutOfBoundsException, но не было");
    } catch (FunctionPointIndexOutOfBoundsException e) {
```

```
        System.out.println("Ловим FunctionPointIndexOutOfBoundsException (index == count): " + e.getMessage());
    }

    //4.проверка нарушений порядка X
    System.out.println("\n4 Проверяем нарушение порядка X:");
    try {
        FunctionPoint bad = new FunctionPoint(100, 0);
        func.setPoint(1, bad);
        System.out.println("Ошибка: ожидалось InappropriateFunctionPointException при setPoint, но не было");
    } catch (InappropriateFunctionPointException e) {
        System.out.println("Ловим InappropriateFunctionPointException при setPoint: " + e.getMessage());
    }

    try {
        func.setPointX(1, func.getPointX(0) -1);
        System.out.println("Ошибка: ожидалось InappropriateFunctionPointException при setPointX, но не было");
    } catch (InappropriateFunctionPointException e) {
        System.out.println("Ловим InappropriateFunctionPointException при setPointX: " + e.getMessage());
    }

    //5. проверка добавления точки с дублирующим X
    System.out.println("\n5 Проверяем добавление точки с повторяющимся X:");
    try {
        double existingX = func.getPointX(2);
```

```
            func.addPoint(new FunctionPoint(existingX, 123));
            System.out.println("Ошибка: ожидалось InappropriateFunctionPointException, но не было");
        } catch (InappropriateFunctionPointException e) {
            System.out.println("Ловим InappropriateFunctionPointException (дубль X): " + e.getMessage());
        }

        //6.проверка удаления точки при < 2 точках
        System.out.println("\n6 Проверяем удаление, если останется <2 точек:");
        try {
            TabulatedFunction small = new ArrayTabulatedFunction(0, 1, new double[]{0, 1});
            System.out.println("Создана функция с " + small.getPointsCount() + " точками");
            small.deletePoint(0);
            System.out.println("Ошибка: ожидалось IllegalStateException, но не было");
        } catch (IllegalStateException e) {
            System.out.println("Ловим IllegalStateException при deletePoint: " + e.getMessage());
        }

        System.out.println("\nПроверка " + func.getClass().getSimpleName() + " завершена успешно!\n");
    }
}
```

Комплексное тестирование обеих реализаций табулированных функций через интерфейс TabulatedFunction. Проверяет вычисление значений, работу с точками и обработку исключений.

Что тестирует:

1. Вычисление значений функции с интерполяцией
2. Доступ к точкам и корректность данных
3. Обработку выхода за границы индексов
4. Проверку порядка координат X
5. Защиту от дублирования X
6. Контроль минимального количества точек

Вывод данного main:

ТЕСТИРОВАНИЕ КЛАССОВ TABULATED FUNCTION

Тест: ArrayTabulatedFunction

Тип функции: ArrayTabulatedFunction

1 Проверяем значения функции:

f(1,0) = 1.0

f(1,5) = 2.5

f(2,0) = 4.0

f(2,5) = 6.5

f(3,0) = 9.0

f(3,5) = 12.5

f(4,0) = 16.0

f(4,5) = NaN

f(5,0) = NaN

2 Проверяем доступ к точкам:

Количество точек: 5

Точка 0: (0,00, 0,00)

Точка 1: (1,00, 1,00)

Точка 2: (2,00, 4,00)

Точка 3: (3,00, 9,00)

Точка 4: (4,00, 16,00)

3 Проверяем ошибки индекса:

Ошибка: ожидалось FunctionPointIndexOutOfBoundsException, но не было

Ловим FunctionPointIndexOutOfBoundsException (index == count): Index 5 out of bounds

4 Проверяем нарушение порядка X:

Ловим InappropriateFunctionPointException при setPoint: X out of order

Ловим InappropriateFunctionPointException при setPointX: X out of order

5 Проверяем добавление точки с повторяющимся X:

Ловим InappropriateFunctionPointException (дубль X): Duplicate X

6 Проверяем удаление, если останется <2 точек:

Создана функция с 2 точками

Ловим IllegalStateException при deletePoint: Cannot delete: less than 3 points

Проверка ArrayTabulatedFunction завершена успешно!

Тест: LinkedListTabulatedFunction

Тип функции: LinkedListTabulatedFunction

1 Проверяем значения функции:

f(1,0) = 1.0

f(1,5) = 2.5

f(2,0) = 4.0

f(2,5) = 6.5

f(3,0) = 9.0

f(3,5) = 12.5

f(4,0) = 16.0

f(4,5) = NaN

f(5,0) = NaN

2 Проверяем доступ к точкам:

Количество точек: 5

Точка 0: (0,00, 0,00)

Точка 1: (1,00, 1,00)

Точка 2: (2,00, 4,00)

Точка 3: (3,00, 9,00)

Точка 4: (4,00, 16,00)

3 Проверяем ошибки индекса:

Ошибка: ожидалось FunctionPointIndexOutOfBoundsException, но не было

Ловим FunctionPointIndexOutOfBoundsException (index == count): Index 5 out of bounds

4 Проверяем нарушение порядка X:

Ловим InappropriateFunctionPointException при setPoint: X out of order

Ловим InappropriateFunctionPointException при setPointX: X out of order

5 Проверяем добавление точки с повторяющимся X:

Ловим InappropriateFunctionPointException (дубль X): Duplicate X

6 Проверяем удаление, если останется <2 точек:

Создана функция с 2 точками

Ловим IllegalStateException при deletePoint: Cannot delete: less than 3 points

Проверка LinkedListTabulatedFunction завершена успешно!

ВСЕ ТЕСТЫ ЗАВЕРШЕНЫ

Process finished with exit code 0

Спасибо за внимание!